Стиль кода и линт

Что проверяется и чем

Инструмент	Версия	Конфигурация	Где запускается
ruff	`ruff==0.17.3` (`requirements-dev.txt`)	`ruff.toml`	`make lint`, CI job «Tests, lint and a runnable app»
pytest	`pytest==8.4.2`	`pytest.ini`, `tests/`	`make test`, CI
gitleaks	`8.28.0` (закреплён в `ci.yml`)	`.gitleaks.toml`	CI job «No committed secrets»

Версии инструментов закреплены в `requirements-dev.txt`, а не ставятся «последними из PyPI»: иначе одна и та же ветка проходит локально и падает в CI (или наоборот) из-за нового набора правил. Набор правил задан в `ruff.toml`, а не берётся по умолчанию — это делает результат одинаковым в редакторе, на машине разработчика и на раннере.

Блокирующий набор правил

`select = ["E9", "F"]` — то, что ломает исполнение и почти всегда указывает на баг: синтаксис (`E9`), неопределённые имена (`F82`), неиспользуемые импорты и переменные (`F401`, `F841`). Всё это в коде исправлено, поэтому проверка проходит с нуля замечаний.

Долг по стилю (осознанно вне блокировки)

Набор	Сколько замечаний	Почему не блокирует
`E4` (несколько импортов в строке)	~50	механическая правка, не влияет на поведение
`E7` (несколько операторов в строке)	~200	тот же легаси-стиль, правка рискованна без тестов на каждый обработчик
`BLE` (широкий `except`)	30+	во Flask-обработчиках широкий перехват осознан; сужать нужно по одному месту
`SIM`	~20	упрощения читаемости, не приоритет

Порядок ужесточения: одно правило → `# noqa: <код>` с комментарием в местах, где поведение намеренное → отдельный PR на сужение исключений → правило в `select`. Массовый `--fix` по всему репозиторию сразу не делается: это ломает `git blame` и мешает разбирать инциденты.

Договорённости, которые не проверяются автоматически

- Новый код — только с тестами; тест не должен требовать сети.
- Фоновые задачи (сокет-события) не пишут в БД в обход `app/user_db.py`.
- Секреты читаются только из окружения, значения по умолчанию — пустые.